

S4-F — WSL Headless TORCS Setup

Contents

WSL headless TORCS — parallel Optuna workers on teacher_wp	1
Milestone A — get ONE headless instance running	1
Milestone B — scale to N workers (after A works)	3
Notes	3

WSL headless TORCS — parallel Optuna workers on teacher_wp

Wersja polska: [README_WSL.pl.md](#)

Goal: run N headless TORCS 1.3.7 instances in WSL Ubuntu, each an Optuna worker joined to the **same** cloud Postgres study teacher_wp, to search the v3 waypoint teacher params faster (target ~80s lap).

We build TORCS from the **same fmirus source** the organizers' Dockerfile uses, so physics == the competition container. We skip Docker/VS Code/Ollama — direct in WSL is lighter and gives us direct access to the S4-F code on /mnt/d.

Race mode is display mode = results only (see practice.xml), so TORCS runs **without 3D rendering** -> CPU-bound, not real-time-bound. It can run *faster* than real-time and scales with cores.

Milestone A — get ONE headless instance running

1. Update apt (in Ubuntu)

```
sudo apt-get update
```

2. Install build + headless deps

```
sudo apt-get install -y --no-install-recommends \
build-essential git wget unzip \
libgl1-mesa-dev libglu1-mesa-dev freeglut3-dev \
libpng-dev libjpeg-dev libopenal-dev libalut-dev libvorbis-dev libogg-dev \
libxi-dev libxmu-dev libxrender-dev libxrandr-dev libxxf86vm-dev libply-dev \
xvfb xautomation python3 python3-pip python3-venv
```

3. Build & install TORCS 1.3.7 (organizers' exact recipe)

```
cd ~
git clone --depth=1 https://github.com/fmirus/torcs-1.3.7.git torcs-src
cd torcs-src
./configure --prefix=/usr/local/torcs
```

```
make # ~20-40 min. Do NOT add -j - the TORCS Makefile races.
sudo make install
sudo make datainstall
```

Put TORCS on PATH:

```
echo 'export PATH=/usr/local/torcs/bin:$PATH' >> ~/.bashrc
export PATH=/usr/local/torcs/bin:$PATH
```

4. Install the SCR server car/driver (from our zip)

```
cp "/mnt/d/IBM_competition/SAC/S3_B/S4-F/TORCS_SETUP/Setup Guide/build your own container/torcs-con
sudo mkdir -p /usr/local/share/games/torcs/drivers
sudo unzip -o ~/scr_server.zip -d /usr/local/share/games/torcs/drivers/
sudo rm -rf /usr/local/share/games/torcs/drivers/__MACOSX
```

5. Sanity check (track + driver present, binary on PATH)

```
ls /usr/local/share/games/torcs/tracks/road/corkscrew # Laguna Seca track
ls /usr/local/share/games/torcs/drivers/scr_server # SCR driver
which torcs # /usr/local/torcs/bin/torcs
```

6. Python env for the driver

```
python3 -m venv ~/torcs-venv
source ~/torcs-venv/bin/activate
pip install --upgrade pip
pip install optuna psycpg2-binary numpy
pip install torch --index-url https://download.pytorch.org/whl/cpu # config.py imports torch (CPU v
echo 'source ~/torcs-venv/bin/activate' >> ~/.bashrc
```

7. Point the driver at the cloud Postgres study

Reads the connection string from your existing .pg_url file (password never typed):

```
echo 'export OPTUNA_STORAGE="$(cat "/mnt/d/IBM_competition/SAC/S3_B/S4-F/.pg_url")"' >> ~/.bashrc
source ~/.bashrc
echo "$OPTUNA_STORAGE" | sed -E 's#://[^@]+@#://***:***@#' # should print a masked postgres URL
```

8. Run ONE worker

```
cd "/mnt/d/IBM_competition/SAC/S3_B/S4-F"
python3 optuna_teacher_linux.py --study-name teacher_wp --n-laps 1 --port 3001
```

Expected: Client connected on 3001 then a stream of [trial N] <lap>s [...vs DAgger] | done=k | <RATE>/hr | best=.... **Report the /hr rate** — it decides how many parallel instances are worth it.

If it hangs on Waiting for server on 3001... torcs -r didn't cooperate with scr_server on your build. Options, in order:

1. Give everything a virtual display (some builds want one even for results-only):

```
xvfb-run -a python3 optuna_teacher_linux.py --study-name teacher_wp --n-laps 1 --port 3001
```

2. Use the proven gym_torcs menu path (Xvfb + xte keystrokes):

```
python3 optuna_teacher_linux.py --study-name teacher_wp --n-laps 1 --port 3001 --launch menu --dis
```

3. Still stuck -> tell me. We watch the menu over VNC (x11vnc -display :1 -nopw &, then a VNC viewer on localhost:5900) and fix the keystroke sequence.

Milestone B — scale to N workers (after A works)

Each worker = its own torcs + own SCR port + own ~/.torcs (so configs don't collide). Rough recipe per extra worker (worker #2 shown):

```
export HOME_ALT=~/.torcs_w2
mkdir -p "$HOME_ALT"
HOME="$HOME_ALT" python3 optuna_teacher_linux.py --study-name teacher_wp --n-laps 1 --port 3002
```

(The driver writes practice.xml under \$HOME/.torcs and patches the scr_server port, so overriding HOME isolates each instance.) **Don't over-provision** — each torcs eats ~1 core; leave headroom. Exact commands finalized once we see A's /hr and your core count.

Notes

- The evaluation (teacher over snakeoil UDP) is identical to Windows; only the TORCS launch differs. Same Optuna study, so optuna_teacher_v3.py --mode report / --mode export-best on Windows still read everything these workers add.
- The native Windows sweep (optuna_teacher_single.py) can keep running in parallel on the same study — Postgres is the shared coordinator.
- torcs -r runs the race then exits, so the driver relauches per trial (cheap on Linux, no menu). This also makes each trial deterministic.